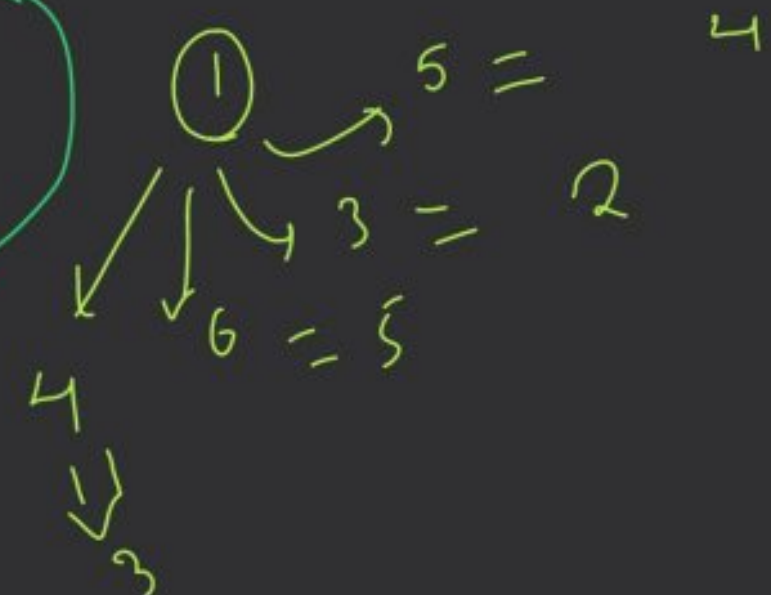


Leetcode 121:- Best Time to Buy & Sell Stock I:-

$[7, 1, 5, 3, 6, 4] = \text{prices}[i]$

$[4, 3, 2, 1] \times$



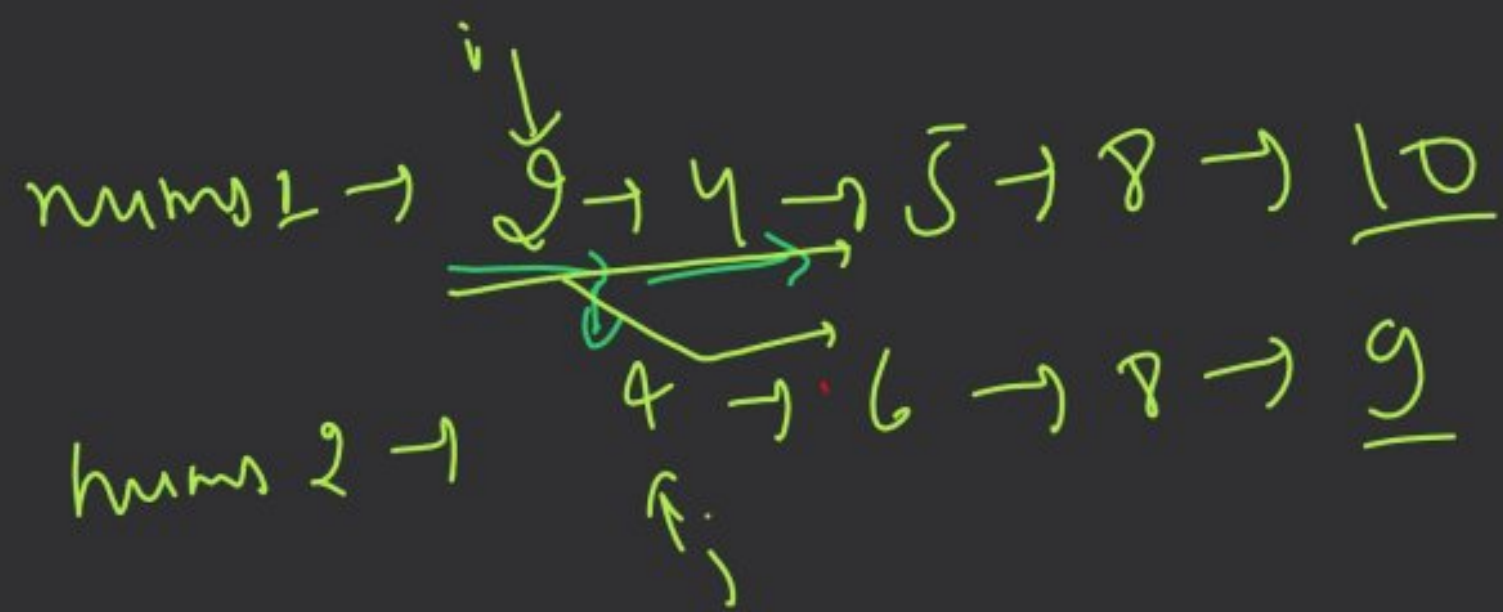
```
int minm = INT_MAX;
int profit = 0;
```

```
for (i = 0 -> n) {
    if (prices[i] < minm) {
        minm = prices[i];
    }
    profit = max(profit, prices[i] - minm);
}
```

$[6, 2, 10, 1, 8, 7, 100]$

minm = INT_MAX, 6, 2, 1
profit = 0, 0, 0, 1, 99

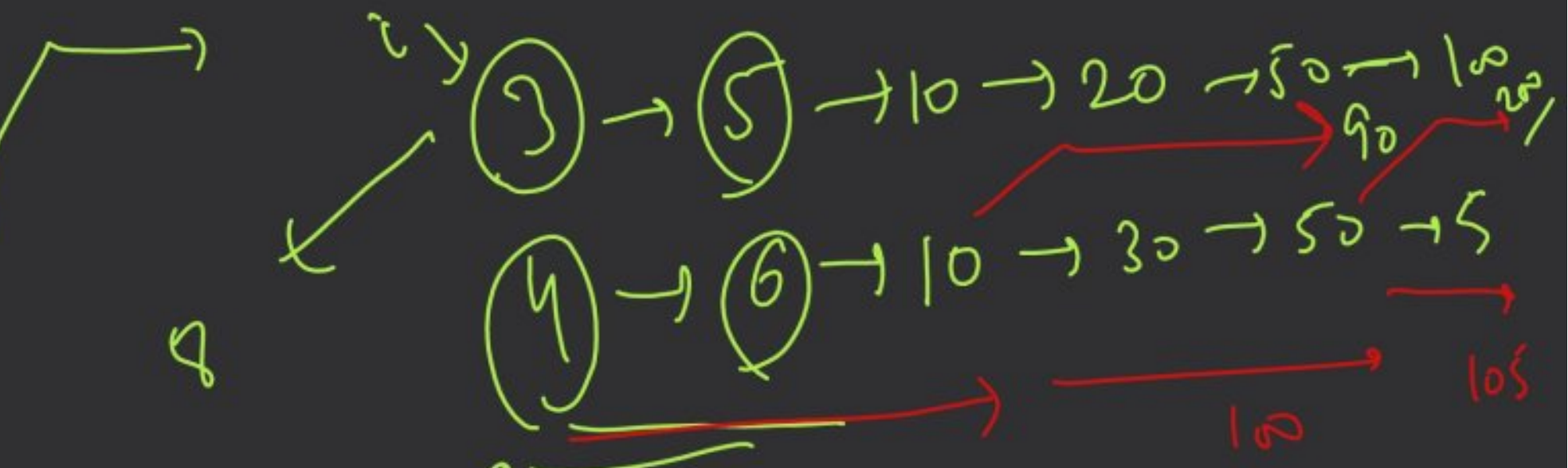
Ques.



S-I :- (Before first common = 4)

nums1 : 2 sum1 = 2
 nums2 : sum2 = 0

At 4, $\max(2, 0) + 4 = 2 + 4 = 6$



S-II :- (Before 8)

nums1 : 5
 nums2 : 6

At 8, $\max(5, 6) + 8 = 14$

S-III :- after 8 :-

nums1 : 10
 nums2 : 9

$\max(10, 9) = 10$

(9, 10)

num 1 = [1, 3, 5, (7, 9)]

num 2 = [1, 3, 5]
 $\uparrow \quad \uparrow \quad \uparrow$
 $\cancel{1} \quad \cancel{3} \quad \cancel{5}$ +X

$$\max(0, 0) + 1 = 0 + 1 = 1$$

$$1 + 3 = 4$$

$$4 + 5 = 9$$

[50, 80, 10, 9, 3, 10, 100]

[20, (100), 10, 200, 10, 10, 1]

80 $\cancel{50} < 100 \checkmark$ Sum1 = ~~50~~ 130
 50 $> 20 \checkmark$ Sum2 = ~~20~~ (120)

(100 > 10)

> < ==

$$\max(120, 120) + 10$$

$\downarrow$
 130 - 110 - 140 $\rightarrow$ result

✓ fail

(Sorting is necessary)

[1, 2, 10, 100] (2 < 50)

[1, 50, 60, 100]

i++
j++
result = 4

[1, 3, 5, 7, 9]

[1, 3, 5]

Sum1 = 2
Sum2 = 5/6/10

Sum1 = 7/8/6

Sum2 = (23) + 12 = 35

```
i = 0, j = 0, sum1 = 0, sum2 = 0, result = 0;
while (i < num1.length && j < num2.length) {
    if (num1[i] < num2[j]) {
        sum1 += num1[i];
        i++;
    }
    else if (num1[i] > num2[j]) {
        sum2 += num2[j];
        j++;
    }
    else {
        result = result + max
            (sum1, sum2) + num1[i];
        sum1 = 0;
        sum2 = 0;
        i++; j++;
    }
}
```


~~[1, 2, 70, 100]~~

70 < 60

(2 < 50)

[⁰1, ¹50, ²60, ³100, ⁴1, 2, 3, 4, 5, 6, 7, 8, 9]

~~i~~ ~~j~~ Sum1 = ~~0~~ ~~2~~ 72

Sum2 = ~~0~~ ~~50~~ 110

res = ~~0~~ ~~2~~ 110

max(Sum1, Sum2) + nums2(i)

res = 110 + 100 = 210

i > j
= = ✓

nums2(j) < nums1(i)

or
nums1(i) > nums2(j)

70 < 60

20 < 100
Sum2

while (i < nums1.length)
{
Sum1 += nums1[i];
while (j < nums2.length)
{
Sum2 += nums2[j];

Majority Element :-

mony ↓

Voeting Element

count, el = nums[0];
= 1;

for (i = 1 → n) {

if (nums[i] == el)

count++;

else

[5, 2, 3, 5, 5, 5, 10, 5, 5, 10]

el = 5 2 3 10 10

count = 1 0 1 2 2 2 2 (4)

[9, 9, 9, 5, 5, 5, 2, 5, 2, 2]

int count = 0, el;

if (count == 0) el = nums[i];

if (nums[i] == el) count++;

else count--;

(2, 5, 5)

[1, 1, 1, 5, 5, 5, 5, 5, 5]

count = 0 1 2 3 2 1 0 3

el = 0 1 (5)

count = 0 1 2 3 2 1 0

el = 0 2 2



Problem List



1:1 Sessions with me



86



Premium

</> Code



Java Auto



```
1 class Solution {
2     public void nextPermutation(int[] nums) {
3         int n = nums.length;
4         int ind = -1;
5
6         // 1. find break point (pivot)
7         for (int i = n - 2; i >= 0; i--) {
8             if (nums[i] < nums[i + 1]) {
9                 ind = i;
10                break;
11            }
12        }
13
14        // 2. if no pivot → reverse whole array
15        if (ind == -1) {
16            reverse(nums, 0, n - 1); ←
17            return;
18        }
19
20        // 3. find next greater element from right
21        for (int i = n - 1; i > ind; i--) {
22            if (nums[i] > nums[ind]) {
23                swap(nums, i, ind); ←
```




Problem List



1:1 Sessions with me



86



Premium

</> Code



Java Auto



```
27
28     // 4. sort/reverse suffix
29     reverse(nums, ind + 1, n - 1); // better than sort
30 }
31
32 // 📩 reverse helper
33 private void reverse(int[] nums, int start, int end) {
34     while (start < end) {
35         swap(nums, start, end);
36         start++;
37         end--;
38     }
39 }
40
41 // 📩 swap helper
42 private void swap(int[] nums, int i, int j) {
43     int temp = nums[i];
44     nums[i] = nums[j];
45     nums[j] = temp;
46 }
47 }
```

$O\left(\frac{n}{2}\right)$



Code

Java Auto

```
1 class Solution {
2     public void nextPermutation(int[] nums) {
3         int n = nums.length;
4         int ind = -1;
5
6         // 1. find break point (pivot)
7         for (int i = n - 2; i >= 0; i--) {
8             if (nums[i] < nums[i + 1]) {
9                 ind = i;
10                break;
11            }
12        }
13
14        // // 2. if no pivot → reverse whole array
15        // if (ind == -1) {
16        //     reverse(nums, 0, n - 1);
17        //     return;
18        // }
19
20        // 3. find next greater element from right
21        for (int i = n - 1; i > ind; i--) {
```

